# Primeira Lista de Exercícios

Tópicos Avançados de Redes Neurais Profundas e Aplicações
Prof. Gilson Antonio Giraldi

Lorran de Araújo Durães Soares

17 de julho de 2026

## Introdução

Este documento apresenta a resolução da primeira lista de exercícios da disciplina *Tópicos Avançados em Redes Neurais Profundas e Aplicações*, ministrada pelo professor Gilson Antonio Giraldi. A lista é composta por três questões, que serão apresentadas ao longo deste documento, seguidas de suas respectivas resoluções.

O código desenvolvido para a realização dos exercícios está disponível no repositório `https://github.com/lorrantwo/LNCC`. A implementação foi desenvolvida em arquivos *Jupyter Notebook* (formato `.ipynb`), utilizando bibliotecas como NumPy [1] para manipulação de tensores, Matplotlib [2] para visualização dos resultados e PyTorch [5] para a definição, treinamento e avaliação dos modelos de inteligência artificial.

## 1 GANs para Parabolóide

**Questão 1:** Considere o parabolóide $P \subset \mathbb{R}^3$ representado por:

$$S : \mathbb{R}^2 \longrightarrow P; \quad S(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2 + 1) \tag{1}$$

Construa o conjunto de dados:

$$D = \{S(x_i^1, x_i^2) \in \mathbb{R}^3; \quad i = 1, \ldots, N\}$$

(a) Usando APENAS o conjunto de dados $D$, gere um modelo GAN que compute $S$;

(b) Tomando o gerador $G$, projete uma estratégia para avaliar a qualidade da representação através do erro quadrático médio (RMSE) e do erro absoluto médio (MAE). Discuta os resultados;

(c) Calcule $\nabla G$ através de diferenciação automática sobre um conjunto de pontos $Z$ no espaço latente. Tomando a expressão (1), calcule $\nabla S$ analiticamente sobre $Z$ e compare os resultados. Discuta seus achados.

**Resolução:**

Primeiramente, para a resolução desta questão, foi gerado o conjunto de dados $D$ a partir do parabolóide definido na Equação 1. Utilizando a biblioteca NumPy [1], foram amostrados 5000 pontos do domínio $\mathcal{X} = [-2, 2] \times [-2, 2]$ a partir de uma distribuição uniforme. Cada ponto $(x_1, x_2) \in \mathcal{X}$ foi mapeado pela função $S(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2 + 1)$, gerando o conjunto de dados $D = \{S(x_i^1, x_i^2)\}_{i=1}^{5000} \subset \mathbb{R}^3$. A Figura 1 ilustra os pontos amostrados no plano cartesiano e a superfície do parabolóide correspondente.

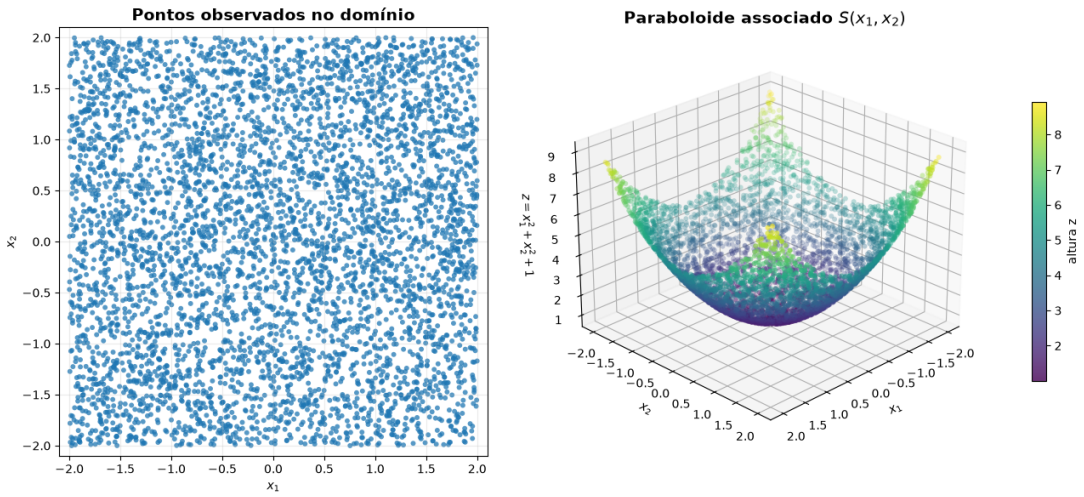



Figura 1: Pontos do domínio $\mathcal{X}$ amostrados uniformemente e o respectivo parabolóide gerado.

Desse conjunto, 80% das amostras foram destinadas ao treinamento da GAN, enquanto os 20% restantes foram utilizados como conjunto de teste. Como o conjunto de dados possui três coordenadas, sendo a terceira definida pela soma dos quadrados das duas primeiras acrescida de uma constante, seus valores tendem naturalmente a apresentar uma escala maior. Para minimizar esse desbalanceamento entre as variáveis de entrada, os dados foram normalizados utilizando a padronização Z-Score:

$$z = \frac{x - \mu}{\sigma}, \tag{2}$$

em que $\mu$ e $\sigma$ representam, respectivamente, a média e o desvio padrão de cada coordenada, calculados apenas sobre o conjunto de treinamento.

Com os dados preparados, foi definida a arquitetura da GAN. A Figura 2 apresenta as arquiteturas da rede Geradora e da rede Discriminadora, definidas empiricamente.

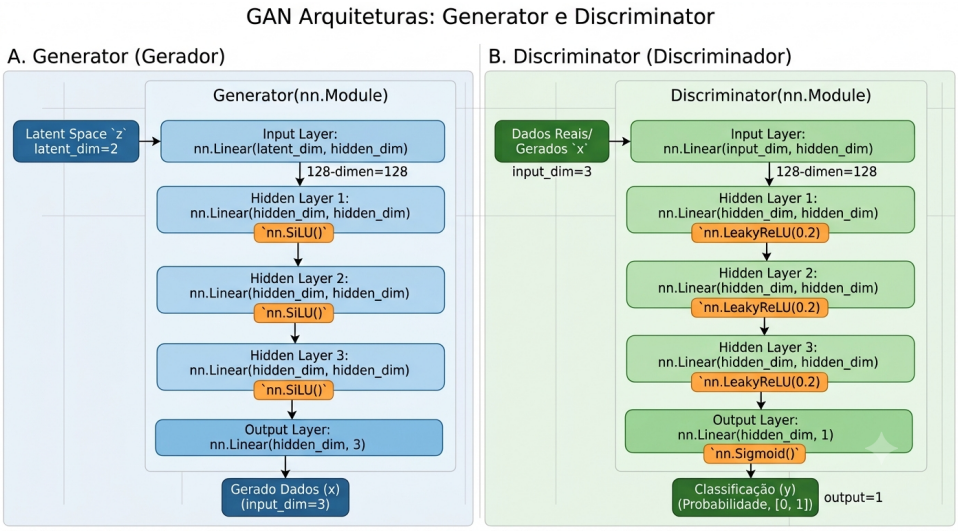



Figura 2: Arquiteturas da rede Geradora e da rede Discriminadora.

A rede Geradora possui cinco camadas totalmente conectadas: uma camada de entrada com dimensão 2, correspondente ao espaço latente, três camadas ocultas com 128 neurônios cada e uma camada de saída

com três neurônios, correspondentes às coordenadas $(x_1, x_2, x_3)$ do parabolóide gerado.

A rede Discriminadora também possui três camadas ocultas com 128 neurônios cada. As camadas intermediárias utilizam a função de ativação LeakyReLU, definida na Equação 3:

$$\text{LeakyReLU}(x) = \max(\alpha x, x), \tag{3}$$

com $\alpha = 0.2$. A camada de saída possui um único neurônio, seguido pela função de ativação sigmoide, definida na Equação 4:

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \tag{4}$$

A saída do discriminador, denotada por $D(x) \in [0, 1]$, representa a probabilidade de uma amostra pertencer ao conjunto de dados reais. Valores próximos de 1 indicam maior confiança de que a amostra seja real, enquanto valores próximos de 0 indicam que ela foi gerada pela rede Geradora.

Para a otimização de ambas as redes foi utilizado o algoritmo Adam, definido pelas Equações 5–8, com taxa de aprendizado igual a $10^{-4}$ e parâmetros $\beta_1 = 0.9$ e $\beta_2 = 0.999$, definidos empiricamente.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \tag{5}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \tag{6}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \tag{7}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \tag{8}$$

em que $g_t$ representa o gradiente da função de custo em relação aos parâmetros $\theta$, $\eta$ é a taxa de aprendizado e $\varepsilon$ é uma constante utilizada para garantir estabilidade numérica.

A função de custo utilizada no treinamento da GAN foi baseada na *Binary Cross-Entropy* (BCE), uma função de erro empregada em problemas de classificação binária. Considerando um rótulo verdadeiro $y \in \{0, 1\}$ e a probabilidade estimada pela rede $\hat{y}$, a BCE é definida como:

$$\mathcal{L}_{BCE}(y, \hat{y}) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]. \tag{9}$$

No treinamento da GAN, o discriminador é responsável por classificar amostras reais como pertencentes à classe $y = 1$ e amostras geradas pelo gerador como pertencentes à classe $y = 0$. Dessa forma, a função de custo do discriminador, obtida a partir da BCE, é dada por:

$$\mathcal{L}_D = -\frac{1}{B} \sum_{i=1}^{B} \left[\log(D(x^{(i)})) + \log(1 - D(G(z^{(i)})))\right], \tag{10}$$

onde $x^{(i)}$ representa uma amostra real do conjunto de treinamento, $z^{(i)}$ representa um vetor amostrado do espaço latente e $B$ é o tamanho do lote.

Por outro lado, durante a atualização da rede Geradora, o objetivo é produzir amostras que sejam classificadas pelo discriminador como reais. Assim, a função de custo do gerador é definida por:

$$\mathcal{L}_G = -\frac{1}{B}\sum_{i=1}^{B}\log(D(G(z^{(i)}))). \tag{11}$$

O treinamento foi realizado durante 300 épocas, utilizando $k = 1$, em que $k$ representa o número de atualizações da rede Discriminadora realizadas para cada atualização da rede Geradora em cada lote (*batch*) de treinamento. Durante o treinamento, a rede Geradora recebe como entrada vetores latentes $z \in \mathbb{R}^d$, amostrados da distribuição uniforme $p_z = U(-2, 2)$. Esses vetores pertencem a um espaço latente independente do espaço dos dados e são utilizados como fonte de aleatoriedade para a geração de novas amostras. A partir desses vetores, a rede Geradora aprende uma transformação capaz de produzir amostras que seguem a distribuição dos dados reais. O procedimento completo de treinamento adotado é apresentado no Algoritmo 1.

**Algorithm 1** Treinamento da GAN

**Require:** Conjunto de treinamento $D$, gerador $G$, discriminador $D$, número de épocas $N$, tamanho do lote $B$, número de atualizações do discriminador $k = 1$

**for** época $= 1$ até $N$ **do**
  **for** cada lote $\{x^{(i)}\}_{i=1}^{B}$ **do**
    **for** $j = 1$ até $k$ **do**
      Amostrar um lote de exemplos reais $\{x^{(i)}\}_{i=1}^{B}$
      Amostrar um lote de vetores latentes $\{z^{(i)}\}_{i=1}^{B} \sim p_z$
      Gerar amostras falsas $\hat{x}^{(i)} = G(z^{(i)})$
      Calcular

$$\mathcal{L}_D = -\frac{1}{B}\sum_{i=1}^{B}\left[\log D(x^{(i)}) + \log\left(1 - D(\hat{x}^{(i)})\right)\right]$$

      Atualizar os parâmetros da rede Discriminadora utilizando Adam
    **end for**
    Amostrar um novo lote de vetores latentes $\{z^{(i)}\}_{i=1}^{B} \sim p_z$
    Gerar amostras falsas $\hat{x}^{(i)} = G(z^{(i)})$
    Calcular

$$\mathcal{L}_G = -\frac{1}{B}\sum_{i=1}^{B}\log D(\hat{x}^{(i)})$$

    Atualizar os parâmetros da rede Geradora utilizando Adam
  **end for**
**end for**

Após o treinamento, a Figura 3 apresenta a evolução da função de custo das redes Geradora e Discriminadora ao longo das épocas. Observa-se que, nas primeiras iterações, a perda da rede Geradora aumenta, indicando que a rede Discriminadora consegue distinguir com facilidade as amostras reais das amostras geradas. À medida que o treinamento evolui, a rede Geradora passa a produzir amostras mais realistas, reduzindo gradualmente sua perda, enquanto a perda da rede Discriminadora tende a estabilizar. Esse comportamento é consistente com o processo esperado de treinamento de uma GAN.

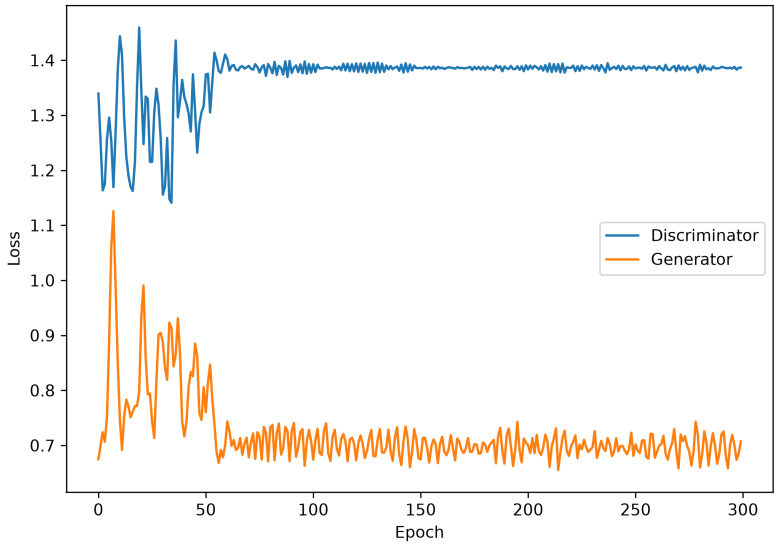



Figura 3: Evolução das funções de perda da rede geradora e da rede discriminadora.

Logo após, o modelo treinado foi utilizado para gerar novas amostras a partir do conjunto de teste. A superfície produzida pela rede Geradora é apresentada na Figura 4. Visualmente, observa-se que a GAN foi capaz de aprender a geometria do parabolóide, produzindo uma superfície bastante semelhante àquela presente no conjunto de dados original.

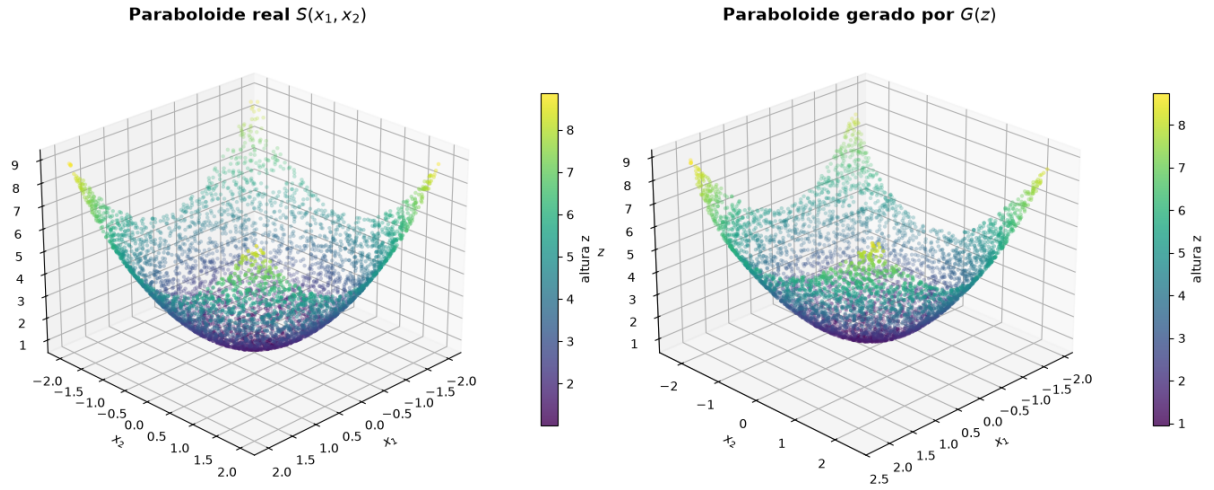



Figura 4: Comparação entre paraboloide do dataset e o gerado pela rede.

Com o intuito de analisar os resultados obtidos, novos vetores do espaço latente foram amostrados e fornecidos à rede Geradora para a criação de novas amostras. Como a GAN aprende a distribuição dos dados e não uma correspondência direta entre entrada e saída, a avaliação não foi realizada comparando pontos individuais gerados com pontos reais, mas sim verificando o erro geométrico das amostras geradas em relação à superfície esperada do parabolóide.

Para isso, foi calculado o erro da superfície gerada utilizando as métricas *Mean Absolute Error* (MAE), definida por:

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|e_i|\,, \tag{12}$$

e *Root Mean Squared Error* (RMSE), definida como:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}e_i^2}, \tag{13}$$

onde $e_i$ representa o erro geométrico entre o ponto gerado e a superfície ideal do parabolóide, dado por:

$$e_i = z_i - (x_i^2 + y_i^2 + 1). \tag{14}$$

Além disso, foram calculadas as versões relativas dessas métricas, normalizando os erros pelo valor médio da superfície gerada. Observa-se que a rede Geradora apresentou um erro geométrico absoluto de aproximadamente 0.35 para a métrica MAE e 0.44 para a métrica RMSE. Quando normalizados, esses valores correspondem a erros relativos de aproximadamente 14.05% e 16.87%, respectivamente. Esses resultados indicam que a GAN foi capaz de aprender a estrutura geral do parabolóide, gerando amostras próximas à superfície real, embora ainda apresente um desvio geométrico devido à dificuldade inerente do treinamento adversarial.

Por fim, considere a parametrização analítica do parabolóide definida pela função $S : \mathbb{R}^2 \to \mathbb{R}^3$,

$$S(x, y) = \left(x, y, x^2 + y^2 + 1\right). \tag{15}$$

Como essa função é diferenciável, seu jacobiano pode ser obtido analiticamente por

$$\nabla S(x, y) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 2x & 2y \end{bmatrix}. \tag{16}$$

Da mesma forma, a rede Geradora define uma transformação $G : \mathcal{Z} \to \mathbb{R}^3$, que recebe vetores do espaço latente $\mathcal{Z}$ e produz pontos no espaço tridimensional. Seu jacobiano pode ser obtido por diferenciação automática, sendo dado por

$$\nabla G(z) = \frac{\partial G(z)}{\partial z}. \tag{17}$$

A comparação entre $\nabla S$ e $\nabla G$ permite investigar se o gerador aprendeu uma parametrização semelhante à função analítica do parabolóide. Caso isso ocorresse, seria esperado que as duas primeiras coordenadas produzidas pelo gerador reproduzissem aproximadamente as coordenadas do vetor latente, enquanto a terceira coordenada representasse a função $x^2 + y^2 + 1$. Entretanto, o treinamento de uma GAN não impõe essa restrição. O objetivo do gerador é apenas transformar vetores do espaço latente em amostras cuja distribuição seja semelhante à distribuição dos dados reais, não existindo qualquer obrigação de preservar as coordenadas do espaço latente durante essa transformação.

A Tabela 1 apresenta a comparação entre os jacobianos da função analítica e da rede Geradora. Para quantificar a diferença entre os jacobianos, foi utilizada a norma de Frobenius da matriz diferença entre

$\nabla G$ e $\nabla S$. Para cada ponto avaliado, essa métrica é definida por

$$\|\nabla G - \nabla S\|_F = \sqrt{\sum_{i=1}^{3}\sum_{j=1}^{2}\left(\frac{\partial G_i}{\partial z_j} - \frac{\partial S_i}{\partial x_j}\right)^2}, \tag{18}$$

em que a norma de Frobenius corresponde à raiz quadrada da soma dos quadrados de todos os elementos da matriz diferença entre os jacobianos. Quanto menor esse valor, maior é a semelhança entre as transformações locais realizadas pela função analítica e pela rede Geradora. Como essa métrica é calculada para cada amostra avaliada, foi considerada a média dessas normas ao longo de todos os pontos analisados, obtendo-se uma medida global da discrepância entre os dois jacobianos.

Observa-se que a norma média de Frobenius da diferença entre os jacobianos foi igual a 2,12, indicando diferenças significativas entre as duas transformações. Em particular, nota-se que as derivadas das duas primeiras coordenadas diferem substancialmente da parametrização analítica. Por exemplo, enquanto para a função $S$ tem-se $\partial x/\partial a = 1$ e $\partial y/\partial b = 1$, a rede Geradora apresentou valores médios próximos de $0,00$ e $-0,03$, respectivamente. De maneira semelhante, as derivadas cruzadas, que são nulas na parametrização analítica, assumiram valores elevados no gerador, como $\partial x/\partial b = -0,93$ e $\partial y/\partial a = -1,14$. Por outro lado, as derivadas associadas à terceira coordenada apresentaram diferenças relativamente menores, com valores médios de $-0,03$ e $-0,24$ para o gerador, enquanto a função analítica apresentou $-0,05$ e $-0,06$.

Tabela 1: Comparação entre os jacobianos da função analítica $S$ e da rede Geradora $G$.

| | $\partial x/\partial a$ | $\partial x/\partial b$ | $\partial y/\partial a$ | $\partial y/\partial b$ | $\partial z/\partial a$ | $\partial z/\partial b$ |
|---|---|---|---|---|---|---|
| $G(a,b)$ | 0.0010 | -0.9281 | -1.1413 | -0.0334 | -0.0333 | -0.2426 |
| $S(a,b)$ | 1.0000 | 0.0000 | 0.0000 | 1.0000 | -0.0546 | -0.0631 |
| $G - S$ | -0.9990 | -0.9281 | -1.1413 | -1.0334 | 0.0213 | -0.1795 |

| Métrica | Valor |
|---|---|
| Norma média de Frobenius | 2.1233 |
| Desvio padrão | 0.1006 |

Esse comportamento pode ser visualizado na Figura 5, que ilustra a correspondência entre os vetores do espaço latente e as duas primeiras coordenadas produzidas pelo gerador. Observa-se que os pontos não permanecem nas mesmas posições do espaço latente, evidenciando que a rede não aprendeu a identidade nas duas primeiras coordenadas. Em vez disso, o gerador aprendeu uma transformação não linear do espaço latente, utilizando os vetores apenas como uma fonte de aleatoriedade para produzir amostras pertencentes à distribuição do parabolóide. Esse resultado está de acordo com o objetivo do treinamento adversarial de uma GAN, que busca reproduzir a distribuição dos dados e não uma parametrização específica da superfície.

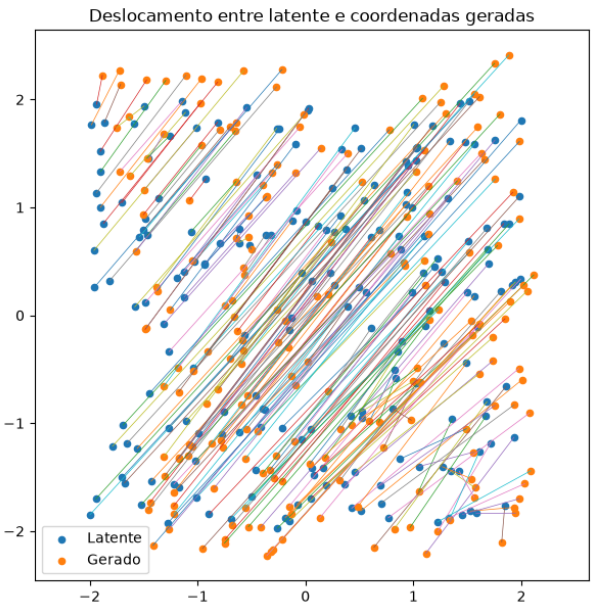



Figura 5: Correspondência entre os vetores do espaço latente e as duas primeiras coordenadas produzidas pela rede Geradora. As setas indicam o deslocamento de cada vetor latente para as coordenadas $(x, y)$ geradas, evidenciando que a rede aprende uma transformação do espaço latente em vez de preservar sua parametrização.

Além desses fatores, deve-se considerar que reproduzir a geometria diferencial da superfície é uma tarefa mais complexa do que apenas reproduzir sua distribuição de pontos. A função objetivo de uma GAN clássica não impõe restrições para que o gerador preserve a estrutura local do *manifold*, como seus gradientes ou espaços tangentes. Dessa forma, mesmo quando as amostras geradas pertencem aproximadamente ao parabolóide, pequenas oscilações locais podem fazer com que os gradientes diferenciem significativamente daqueles da superfície analítica [4].

## 2 GANs com Operações Convolucionais

**Questão 2:** Escolha uma base de dados de imagens e treine um modelo GAN sobre ela, com um gerador utilizando operações convolucionais.

(a) Considere dois critérios de parada diferentes e compare-os;

(b) Escolha duas métricas de qualidade de imagem e compare-as para analisar o resultado da GAN.

**Resolução:**

Para a realização deste experimento, foi utilizado o conjunto de dados *MNIST* (*Modified National Institute of Standards and Technology*) [3], composto por imagens em escala de cinza de dígitos manuscritos. Cada imagem possui dimensão de $28 \times 28$ pixels e pertence a uma das dez classes correspondentes aos dígitos de 0 a 9. O conjunto de dados contém 70.000 imagens, sendo 60.000 destinadas ao treinamento e 10.000 ao teste. Neste trabalho, apenas o conjunto de treinamento foi utilizado para o treinamento da Rede Generativa Adversarial (GAN). A Figura 6 apresenta alguns exemplos de imagens presentes na base de dados. Além disso, através da Figura 7, podemos observar que o conjunto de dados é aproximadamente balanceado em relação ao número de amostras de cada classe disponíveis para treinamento.

Como a camada de saída da rede geradora utilizará a função de ativação *Tanh*, cujos valores pertencem ao intervalo $[-1, 1]$, as imagens do MNIST, originalmente representadas no intervalo $[0, 1]$, foram

normalizadas para o intervalo $[-1, 1]$ por meio da seguinte transformação:

$$x_{\text{norm}} = 2x - 1. \tag{19}$$

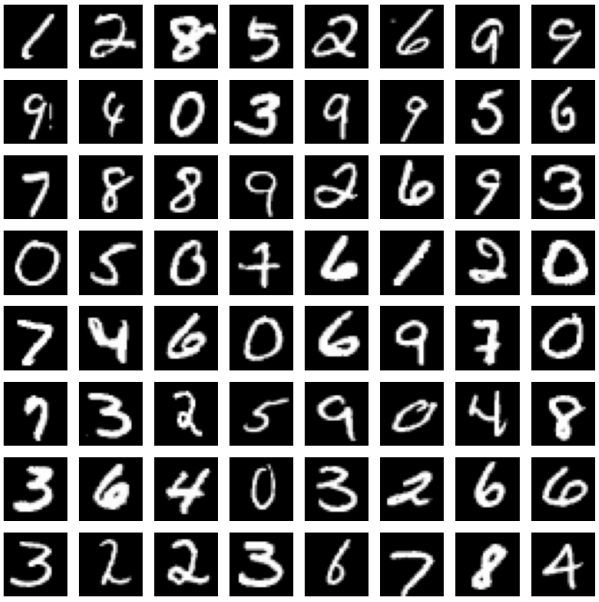


Figura 6: Exemplos de imagens do conjunto de treinamento do MNIST.

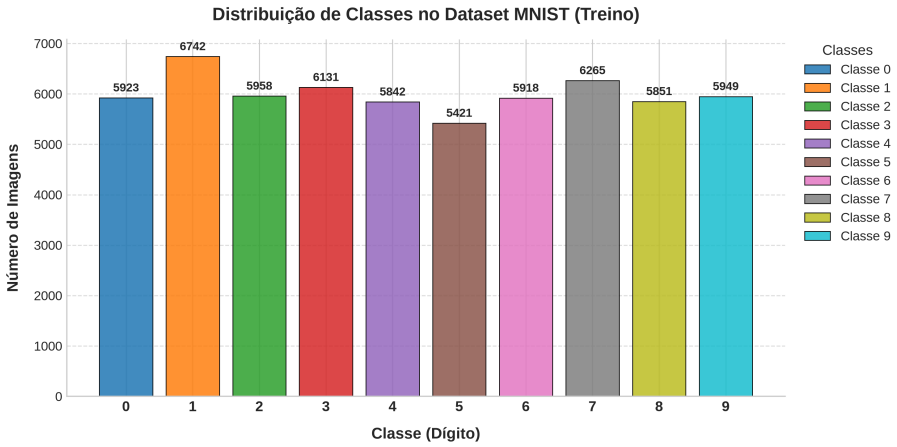



Figura 7: Distribuição dos dados de treinamento por classes.

Considerando que o problema envolve a geração de imagens, foi adotada uma arquitetura convolucional para a GAN, inspirada na estrutura proposta pelas DCGANs (*Deep Convolutional Generative Adversarial Networks*). A arquitetura utilizada para as redes é ilustrada na Figura 8.

Como podemos observarm, a rede geradora recebe como entrada um vetor de ruído aleatório de dimensão 100, representado no formato $(100, 1, 1)$. A partir desse vetor, sucessivas camadas de convolução transposta (*ConvTranspose2d*) realizam o aumento gradual da resolução espacial até produzir uma imagem sintética de dimensão $28 \times 28$. Entre as camadas convolucionais são empregadas camadas de *Batch Normalization* e a função de ativação ReLU, enquanto a camada de saída utiliza a função *Tanh*, garantindo que os valores gerados permaneçam no mesmo intervalo dos dados normalizados.

Por outro lado, a rede discriminadora recebe como entrada imagens reais ou sintéticas de dimensão $28 \times 28$ e realiza sucessivas operações de convolução para reduzir sua dimensão espacial e extrair características discriminativas. Nas camadas intermediárias é utilizada a função de ativação *LeakyReLU* com coeficiente negativo igual a 0,2, além de uma camada de *Batch Normalization*. A camada final produz um único valor escalar, indicando o grau de confiança da rede de que a imagem fornecida pertence ao conjunto de dados reais.

A Tabela 2 resume a arquitetura completa das redes geradora e discriminadora implementadas neste trabalho. Os hiperparâmetros da arquitetura foram definidos empiricamente, a partir de experimentos envolvendo diferentes configurações de número de filtros, tamanhos de kernel e demais parâmetros das camadas convolucionais.

Tabela 2: Arquitetura da GAN utilizada nos experimentos.

| Rede | Camada | Filtros | Kernel | Stride | Padding |
|---|---|---|---|---|---|
| Geradora | ConvTranspose2d | 256 | $7 \times 7$ | 1 | 0 |
| | ConvTranspose2d | 128 | $4 \times 4$ | 2 | 1 |
| | ConvTranspose2d | 1 | $4 \times 4$ | 2 | 1 |
| Discriminadora | Conv2d | 64 | $4 \times 4$ | 2 | 1 |
| | Conv2d | 128 | $4 \times 4$ | 2 | 1 |
| | Conv2d | 1 | $7 \times 7$ | 1 | 0 |

Na rede geradora, as duas primeiras camadas convolucionais transpostas são seguidas por operações de *Batch Normalization* e pela função de ativação ReLU. Já na camada de saída é utilizada a função *Tanh*. Na rede discriminadora, a primeira camada convolucional é seguida por uma função *LeakyReLU*; a segunda camada utiliza *Batch Normalization* e *LeakyReLU*; enquanto a camada final não emprega função de ativação, produzindo diretamente o escore utilizado pela função de perda durante o treinamento.

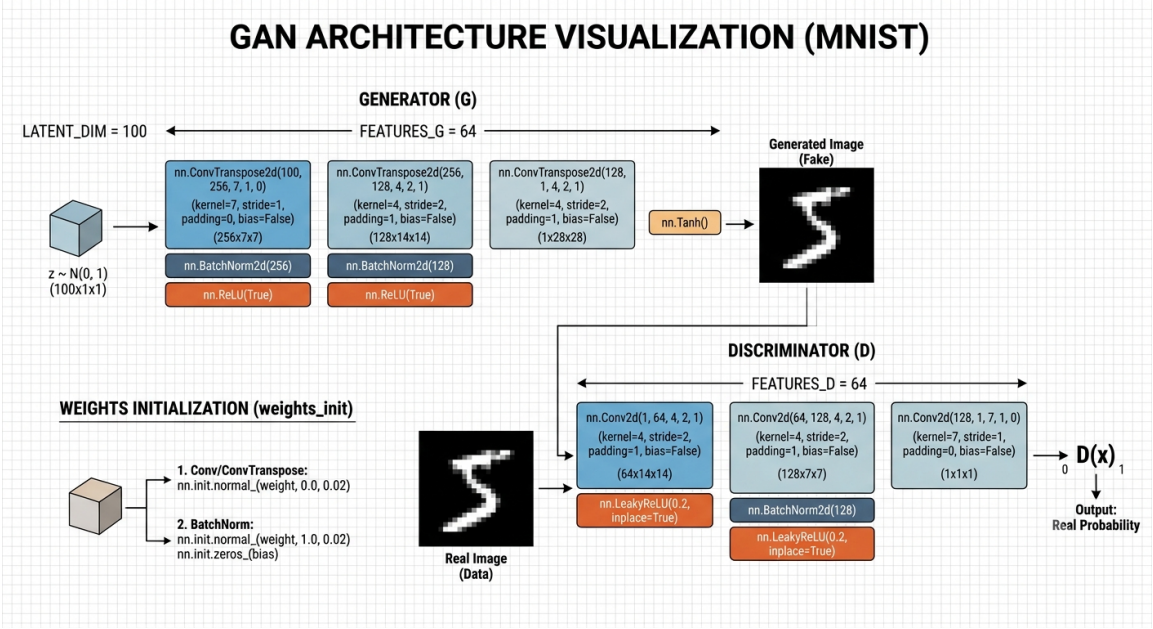



Figura 8: Arquitetura da GAN convolucional utilizada neste trabalho.

O treinamento foi realizado de maneira análoga ao descrito na Questão 1, utilizando o otimizador Adam com os mesmos hiperparâmetros e as mesmas funções de custo para o gerador e o discriminador.

Considerando um limite máximo de 100 épocas, foram avaliadas duas estratégias distintas de parada antecipada, descritas a seguir.

- **(M1) Early Stopping baseado no FID:** nesta estratégia, a qualidade das imagens geradas foi monitorada por meio da métrica *Fréchet Inception Distance* (FID), definida por

$$\mathrm{FID}(\mathbf{x}, \mathbf{g}) = \left\|\boldsymbol{\mu}_x - \boldsymbol{\mu}_g\right\|_2^2 + \mathrm{Tr}\left(\Sigma_x + \Sigma_g - 2(\Sigma_x \Sigma_g)^{1/2}\right), \tag{20}$$

  onde $(\boldsymbol{\mu}_x, \Sigma_x)$ representam a média e a matriz de covariância das características extraídas das imagens reais e $(\boldsymbol{\mu}_g, \Sigma_g)$ representam as estatísticas correspondentes das imagens geradas. Quanto menor o valor do FID, maior a similaridade entre as distribuições das imagens reais e sintéticas.

  O treinamento era interrompido caso o melhor valor de FID não apresentasse melhoria durante uma janela consecutiva de $n = 10$ épocas.

- **(M2) Estabilidade da acurácia do discriminador:** nesta estratégia, foi monitorada a acurácia do discriminador sobre o conjunto de validação ao final de cada época. A hipótese adotada é que, quando essa acurácia passa a oscilar dentro de uma faixa reduzida por um período suficientemente longo, o treinamento pode ter atingido um estado próximo do equilíbrio entre gerador e discriminador.

  Para verificar essa condição, foi considerada uma janela móvel contendo as últimas 10 épocas. O treinamento era encerrado quando a diferença entre a maior e a menor acurácia observadas nessa janela fosse inferior a 10%, caracterizando um comportamento suficientemente estável do discriminador.

Após definir os critérios de parada, a GAN foi treinada utilizando cada uma das estratégias propostas. A Figura 9 apresenta a evolução das funções de custo normalizadas do gerador e do discriminador durante o treinamento. Observa-se que a estratégia M1 interrompeu o treinamento na época 26, momento em que o critério de estagnação do FID foi satisfeito. Em contrapartida, utilizando a estratégia M2, o treinamento prosseguiu até a época 59, quando foi detectada a estabilidade da acurácia do discriminador.

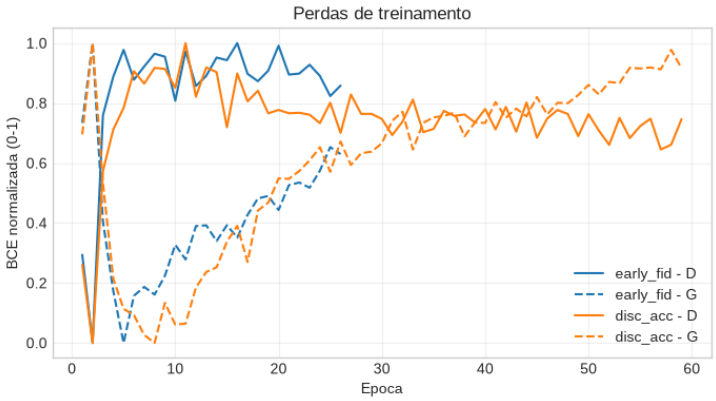



Figura 9: Evolução das funções de custo normalizadas do gerador e do discriminador durante o treinamento da GAN para as duas estratégias de parada antecipada avaliadas: Early FID (M1) e estabilidade da acurácia do discriminador (M2). Observa-se que o treinamento utilizando M1 foi interrompido na época 26, enquanto o treinamento utilizando M2 prosseguiu até a época 59.

Para analisar o comportamento de ambos os critérios, a Figura 10 apresenta a evolução do FID ao longo das épocas, enquanto a Figura 11 mostra a evolução da acurácia do discriminador.

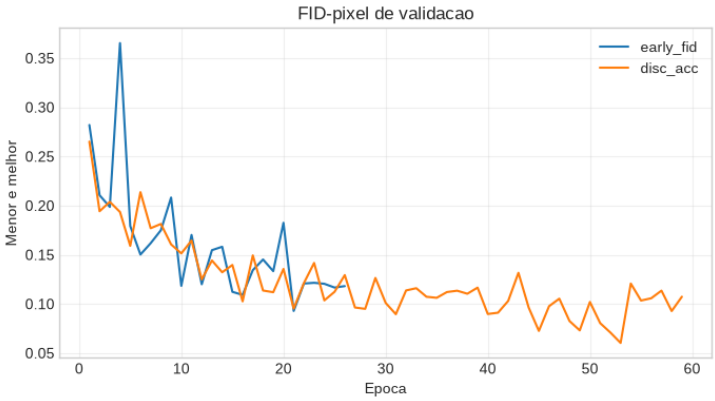



Figura 10: Evolução da métrica Fréchet Inception Distance (FID) ao longo do treinamento para as estratégias Early FID (M1) e estabilidade da acurácia do discriminador (M2). Valores menores indicam maior similaridade entre as distribuições das imagens reais e geradas.

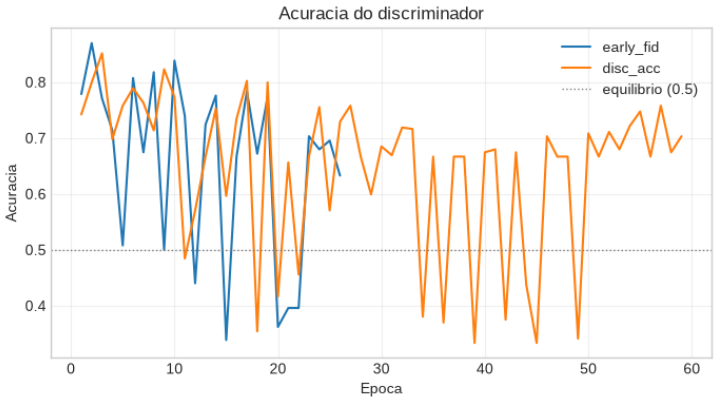


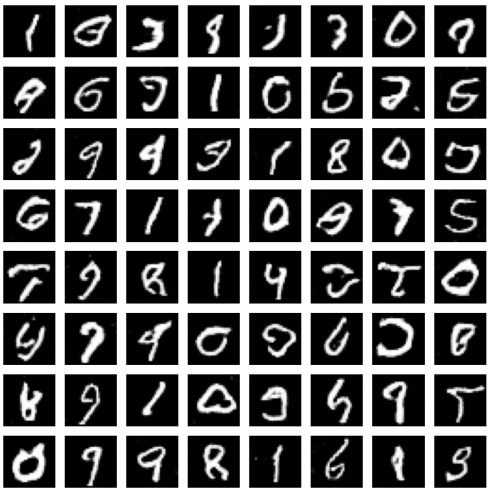

Figura 11: Evolução da acurácia do discriminador no conjunto de validação durante o treinamento da GAN para as estratégias Early FID (M1) e estabilidade da acurácia do discriminador (M2). A região final da curva de M2 apresenta menor variação entre épocas, satisfazendo o critério de estabilidade utilizado para a parada antecipada.

Observando a Figura 10, nota-se que a estratégia M1 interrompeu o treinamento quando o FID permaneceu sem melhora durante a janela de monitoramento estabelecida. Entretanto, ao analisar a curva correspondente ao treinamento utilizando M2, percebe-se que, após algumas épocas adicionais, o FID voltou a diminuir e atingiu valores inferiores ao obtido pela estratégia M1. Esse comportamento indica que, para os hiperparâmetros utilizados, o critério de parada baseado exclusivamente na estagnação do FID mostrou-se excessivamente conservador, interrompendo o treinamento antes que o gerador atingisse sua melhor qualidade.

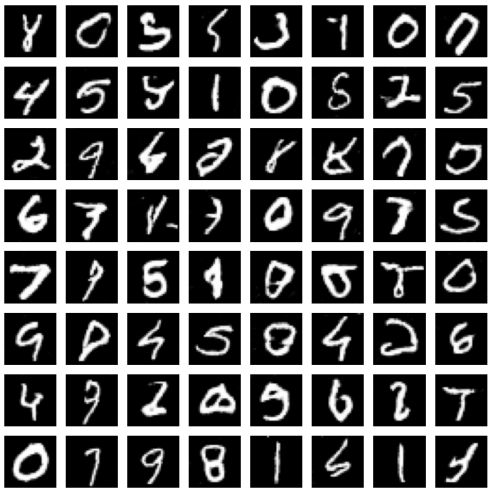
A Figura 11 permite interpretar o comportamento da estratégia M2. Entre aproximadamente as épocas 30 e 50, a acurácia do discriminador apresentou oscilações relativamente elevadas, indicando que o processo adversarial ainda estava em evolução. Somente após esse período a acurácia passou a permanecer dentro da faixa de estabilidade estabelecida, levando ao encerramento do treinamento. Embora os valores absolutos da acurácia nesse momento sejam relativamente elevados, sugerindo que o discriminador ainda conseguia distinguir parte das imagens sintéticas, a evolução do FID demonstra que o gerador continuou melhorando sua capacidade de produzir amostras realistas durante esse intervalo.

A Figura 12 apresenta exemplos de imagens sintetizadas pelos modelos obtidos através das duas estratégias de parada. Visualmente, observa-se uma ligeira superioridade das imagens produzidas pela estratégia

M2, que apresentam menor quantidade de artefatos no fundo e dígitos com formas mais consistentes.



(a) Imagens geradas pelo modelo treinado utilizando o critério de parada antecipada baseado no FID (M1).



(b) Imagens geradas pelo modelo treinado utilizando o critério de estabilidade da acurácia do discriminador (M2).

Figura 12: Exemplos de dígitos sintéticos produzidos pelos modelos obtidos utilizando os dois critérios de parada antecipada. Visualmente, observa-se que o modelo treinado com o critério M2 produz amostras mais consistentes, com menor quantidade de artefatos e melhor definição dos dígitos.

Como referência adicional, o modelo também foi treinado durante as 100 épocas previstas, sem a utilização de parada antecipada. A Tabela 3 resume os resultados quantitativos obtidos utilizando as métricas FID e MMD para os três modelos treinados.

Tabela 3: Comparação entre os diferentes critérios de parada.

| **Estratégia** | **Épocas** | **FID ↓** |
|---|---|---|
| Treinamento completo | 100 | 0.1075 |
| Early FID (M1) | 26 | 0.1164 |
| Estabilidade do discriminador (M2) | 59 | **0.0727** |

Os resultados quantitativos corroboram a análise visual. Apesar de o treinamento completo utilizar todas as 100 épocas disponíveis, ele não apresentou o melhor desempenho. A estratégia M2 obteve simultaneamente o menor valor de FID e de MMD, indicando maior proximidade entre as distribuições das imagens reais e geradas. Por outro lado, a estratégia M1 apresentou o pior desempenho entre os três cenários, reforçando que, para esta configuração experimental, o critério de parada baseado apenas na estagnação do FID interrompeu o treinamento prematuramente.

De maneira geral, os experimentos indicam que a utilização da estabilidade da acurácia do discriminador como critério de parada mostrou-se uma alternativa mais eficiente para este problema, permitindo que o gerador continuasse evoluindo mesmo após períodos temporários de estagnação do FID e produzindo, ao final do treinamento, amostras de melhor qualidade.

# 3 CNN para Estimativa de Idade

**Questão 3:** Utilize a base de dados UTKFace (`https://github.com/aicip/UTKFace`) ou outro repositório similar, e considere o problema de regressão de estimar a idade dada uma imagem facial usando

um modelo CNN. Aplique teoria de invariância (ou equivariância) para melhorar os resultados.

**Resolução:**

Na realização desta questão foi utilizado o conjunto de dados *UTKFace*, composto por 23.708 imagens RGB de faces humanas com diferentes idades, gêneros e etnias. Cada imagem possui como rótulo a idade da pessoa retratada, permitindo que o problema seja tratado como uma tarefa de regressão.

A Figura 13 apresenta a distribuição das amostras por faixas etárias. Observa-se um desbalanceamento significativo na base de dados, com uma elevada concentração de exemplos entre 20 e 29 anos, enquanto as faixas etárias mais elevadas, especialmente acima dos 80 anos, possuem uma quantidade substancialmente menor de amostras. Esse desbalanceamento pode influenciar diretamente o processo de aprendizado da rede, uma vez que modelos supervisionados tendem a apresentar melhor capacidade de generalização para regiões do espaço de dados mais representadas durante o treinamento. Consequentemente, espera-se um desempenho superior nas faixas etárias com maior número de exemplos e maiores erros para idades pouco representadas.

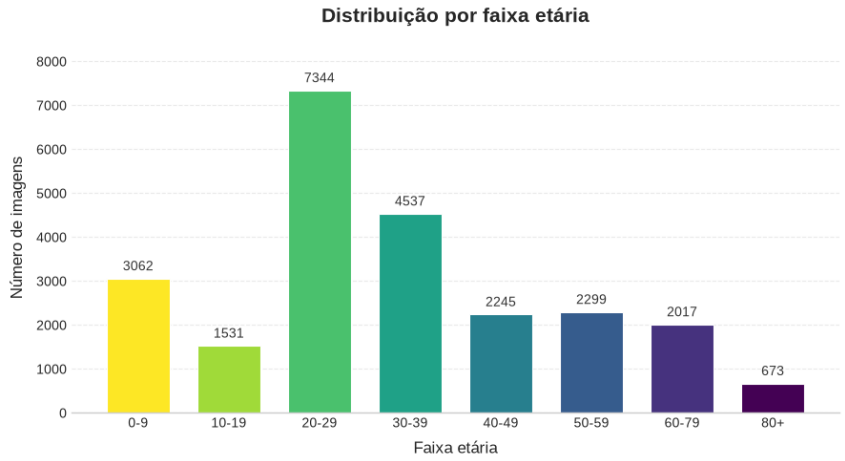



Figura 13: Distribuição do número de imagens do conjunto UTKFace agrupadas por faixas etárias. Observa-se uma concentração de amostras em adultos jovens, principalmente entre 20 e 29 anos, enquanto as faixas etárias mais avançadas apresentam quantidade significativamente menor de exemplos.

O conjunto de dados foi dividido em três subconjuntos independentes, sendo 16.595 imagens destinadas ao treinamento, 1.556 imagens para validação e 3.557 imagens para teste. O conjunto de treinamento foi utilizado para o ajuste dos parâmetros da rede neural, enquanto o conjunto de validação foi empregado para monitorar o desempenho do modelo durante o treinamento e selecionar o melhor ponto de parada. Por fim, o conjunto de teste foi reservado exclusivamente para a avaliação final do modelo, garantindo uma estimativa imparcial de sua capacidade de generalização.

Antes de serem fornecidas à rede, todas as imagens foram normalizadas canal a canal segundo

$$x' = \frac{x - \mu}{\sigma}, \tag{21}$$

em que $\mu = 0{,}5$ e $\sigma = 0{,}5$ para os três canais RGB. Como os pixels já estavam representados no intervalo $[0, 1]$, essa transformação mapeia seus valores para o intervalo $[-1, 1]$, tornando a distribuição dos dados mais adequada ao treinamento da rede neural e favorecendo a estabilidade do processo de otimização.

Como modelo de regressão foi utilizada uma rede neural convolucional (CNN), cuja arquitetura é composta por dois módulos principais. O primeiro consiste em uma sequência de blocos convolucionais

responsáveis pela extração automática de características das imagens de entrada. Cada bloco é formado por uma camada convolucional, seguida por uma função de ativação ReLU e operações de redução da dimensionalidade espacial, permitindo que a rede aprenda representações hierárquicas das faces.

Após a etapa de extração de características, é aplicado um operador de *Adaptive Average Pooling*, que reduz os mapas de características para um vetor de tamanho fixo, independentemente das dimensões espaciais da entrada. Em seguida, esse vetor é processado por camadas totalmente conectadas (*fully connected*), responsáveis por realizar a tarefa de regressão e estimar a idade da pessoa presente na imagem. Para reduzir o sobreajuste durante o treinamento, foi empregada uma camada de *dropout* com taxa de 30%.

A Figura 14 apresenta a arquitetura utilizada neste trabalho, destacando os blocos convolucionais responsáveis pela extração de características e as camadas lineares responsáveis pela predição da idade.

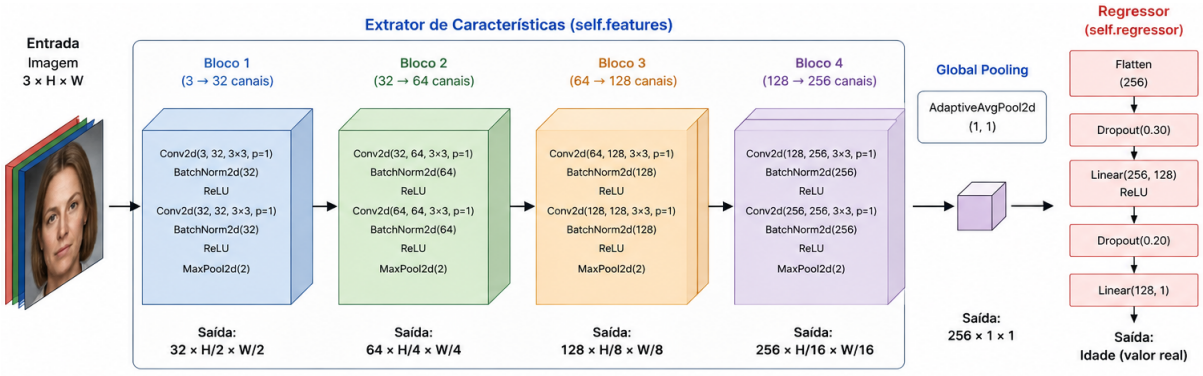



Figura 14: Arquitetura da rede neural convolucional utilizada para a regressão da idade. O modelo é composto por quatro blocos convolucionais para extração de características, seguidos por uma etapa de *Adaptive Average Pooling* e camadas totalmente conectadas que produzem uma única saída correspondente à idade estimada.

O treinamento foi realizado de maneira análoga às questões anteriores, utilizando o otimizador Adam com seus hiperparâmetros padrão para minimizar a função de custo do modelo.

Seja $f_\theta$ a rede neural parametrizada pelo conjunto de parâmetros $\theta$. Considerando um mini-lote de treinamento $B$, a função de custo utilizada corresponde ao Erro Quadrático Médio (*Mean Squared Error – MSE*), definido por

$$\mathcal{L}_{\text{MSE}} = \frac{1}{|B|} \sum_{i=1}^{|B|} \left(y_i - f_\theta(x_i)\right)^2, \tag{22}$$

em que $x_i$ representa a $i$-ésima imagem do mini-lote, $y_i$ corresponde à idade real associada a essa imagem e $f_\theta(x_i)$ é a idade estimada pela rede neural. Essa função penaliza quadraticamente erros de predição, atribuindo maior peso a estimativas mais distantes do valor real, sendo amplamente empregada em problemas de regressão.

Para explorar o princípio da invariância durante o treinamento da rede neural, foram avaliadas duas estratégias distintas baseadas na aplicação de transformações às imagens de entrada.

- **Aumentação dos dados:** nesta abordagem, cada imagem de entrada $x_i$ pode ser submetida, de forma aleatória, a um conjunto de transformações antes de ser apresentada à rede. As transformações utilizadas foram:

- **Recorte aleatório com redimensionamento (*Random Resized Crop*):** realiza um recorte aleatório preservando entre 85% e 100% da área original da imagem, permitindo pequenas variações na razão de aspecto antes do redimensionamento para o tamanho de entrada da rede.
- **Espelhamento horizontal aleatório (*Random Horizontal Flip*):** inverte horizontalmente a imagem com probabilidade de 50%, aumentando a variabilidade das poses sem alterar a idade da pessoa.
- **Rotação aleatória (*Random Rotation*):** aplica rotações aleatórias de até $\pm 10°$, tornando o modelo mais robusto a pequenas inclinações da cabeça.
- **Variação de brilho, contraste, saturação e tonalidade (*Color Jitter*):** modifica levemente as características fotométricas da imagem, simulando diferentes condições de iluminação e aquisição.

Seja $\mathcal{T}$ uma transformação pertencente ao conjunto descrito anteriormente. Nesse caso, a função de custo passa a ser dada por

$$\mathcal{L}_{\text{MSE}} = \frac{1}{|B|} \sum_{i=1}^{|B|} \left(y_i - f_\theta(\mathcal{T}(x_i))\right)^2 . \tag{23}$$

A ideia central é que a idade estimada pela rede permaneça aproximadamente a mesma independentemente da transformação aplicada à imagem. Assim, deseja-se que

$$f_\theta(\mathcal{T}_1(x_i)) \approx f_\theta(\mathcal{T}_2(x_i)) \approx \cdots \approx f_\theta(\mathcal{T}_n(x_i)) \approx f_\theta(x_i) \approx y_i, \tag{24}$$

em que $\mathcal{T}_1, \ldots, \mathcal{T}_n$ representam diferentes transformações possíveis. Como essas transformações são aplicadas aleatoriamente durante o treinamento, uma mesma imagem pode sofrer diferentes modificações em épocas distintas ou até mesmo permanecer inalterada.

- **Regularização de Consistência:** diferentemente da abordagem anterior, nesta estratégia a imagem original continua sendo utilizada na função de custo supervisionada principal. Entretanto, uma versão transformada da mesma imagem é utilizada para construir um termo adicional de regularização que incentiva a rede a produzir predições consistentes para ambas as versões.

  A função de custo torna-se

$$\mathcal{L} = \frac{1}{|B|} \sum_{i=1}^{|B|} \left(y_i - f_\theta(x_i)\right)^2 + \lambda \frac{1}{|B|} \sum_{i=1}^{|B|} \left(f_\theta(\mathcal{T}(x_i)) - f_\theta(x_i)\right)^2 , \tag{25}$$

  em que o primeiro termo corresponde ao erro de regressão supervisionado e o segundo representa a perda de consistência. O hiperparâmetro $\lambda$ controla a contribuição desse termo durante o treinamento e, neste trabalho, foi fixado em $\lambda = 0,5$.

  Dessa forma, além de minimizar o erro em relação ao rótulo verdadeiro, o modelo é explicitamente incentivado a produzir estimativas semelhantes para uma imagem e suas diferentes versões transformadas, tornando a rede mais robusta às variações presentes nos dados.

A partir dessas estratégias, foram considerados quatro cenários de treinamento, todos executados durante 50 épocas:

- **M1:** treinamento convencional, sem aumentação de dados e sem regularização de consistência;

- **M2:** treinamento utilizando apenas aumentação de dados;
- **M3:** treinamento utilizando apenas regularização de consistência;
- **M4:** treinamento utilizando simultaneamente aumentação de dados e regularização de consistência.

A Figura 15 apresenta a evolução da função de custo durante o treinamento para os quatro modelos considerados. Observa-se que os modelos M1 e M2 alcançam menores valores da função de custo ao longo das épocas quando comparados aos modelos M3 e M4. Entretanto, essa comparação deve ser feita com cautela, uma vez que os modelos M3 e M4 incluem um termo adicional de regularização por consistência na função objetivo. Dessa forma, a perda total desses modelos não representa apenas o erro de regressão, mas também a penalização associada à consistência das predições, tornando seus valores naturalmente mais elevados.

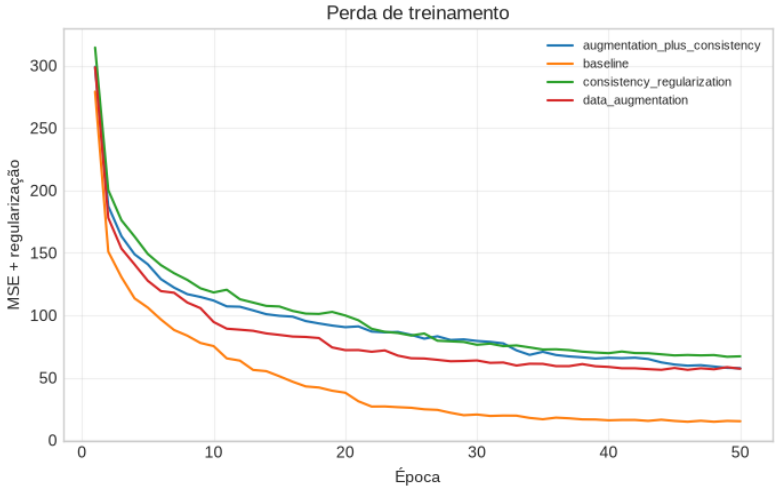



Figura 15: Evolução da função de custo durante o treinamento dos quatro modelos avaliados. Os modelos M3 e M4 apresentam valores mais elevados devido à inclusão do termo de regularização por consistência na função objetivo.

Por outro lado, a Figura 16 apresenta a evolução do Erro Absoluto Médio (MAE, Equação 12) calculado sobre o conjunto de validação. Observa-se que todos os modelos apresentam oscilações relativamente elevadas durante as primeiras épocas de treinamento, comportamento esperado enquanto a rede ainda está convergindo. Após aproximadamente a época 30, as curvas tornam-se mais estáveis, permitindo uma comparação mais consistente entre os métodos. Nota-se que o modelo M4 alcança os menores valores de MAE ao final do treinamento, enquanto o modelo convencional (M1) apresenta o pior desempenho entre as abordagens avaliadas.

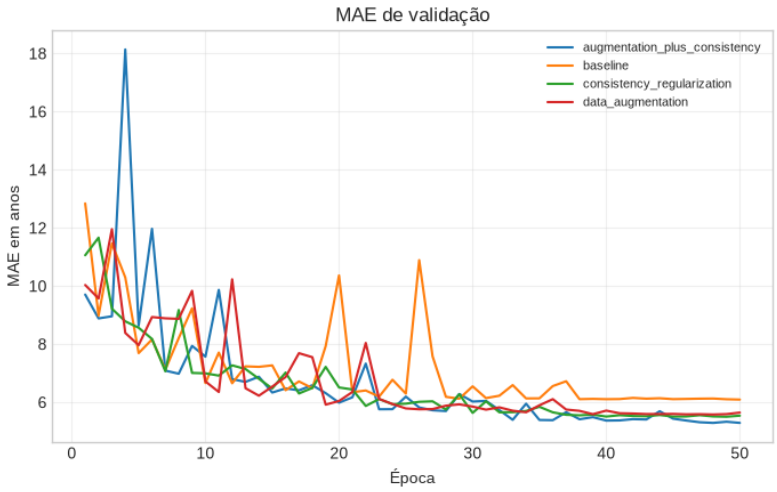



Figura 16: Evolução do Erro Absoluto Médio (MAE) no conjunto de validação durante o treinamento dos modelos M1 (baseline), M2 (aumentação de dados), M3 (regularização de consistência) e M4 (combinação das duas estratégias). Observa-se uma redução progressiva do erro ao longo das épocas, com o modelo M4 apresentando os menores valores ao final do treinamento.

A Figura 17 apresenta os valores finais de MAE e RMSE (Equação 13) obtidos sobre o conjunto de teste para cada modelo. Os resultados mostram que o modelo M4 obteve simultaneamente os menores valores para ambas as métricas, indicando maior precisão na estimativa da idade das pessoas. Em contrapartida, o modelo M1 apresentou os maiores erros, evidenciando o benefício da incorporação do princípio da invariância durante o treinamento.

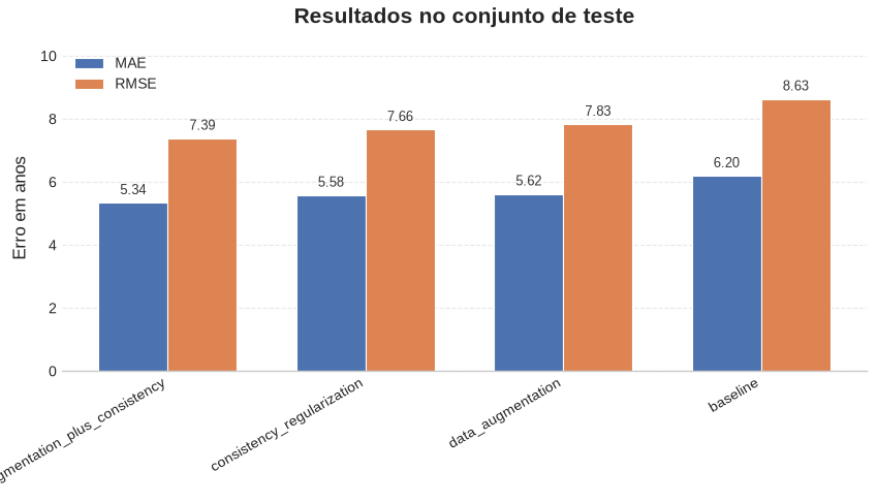



Figura 17: Valores finais de MAE e RMSE obtidos no conjunto de teste para os quatro modelos avaliados. O modelo M4 apresenta os menores valores para ambas as métricas, indicando melhor desempenho na tarefa de regressão da idade.

A Tabela **??** resume quantitativamente os resultados obtidos para os quatro modelos, incluindo os valores absolutos de MAE e RMSE, bem como a melhoria percentual em relação ao modelo baseline (M1). Observa-se que tanto a aumentação de dados quanto a regularização de consistência contribuem para reduzir os erros de predição, sendo a combinação das duas estratégias a que proporciona o melhor desempenho geral.

Por fim, a Figura 18 apresenta exemplos de imagens pertencentes ao conjunto de teste, acompanhadas

da idade real e das idades estimadas pelos modelos M1, M2, M3 e M4. De maneira geral, observa-se que os modelos que incorporam o princípio da invariância produzem estimativas mais próximas dos valores reais, especialmente o modelo M4, que apresenta maior consistência nas predições.

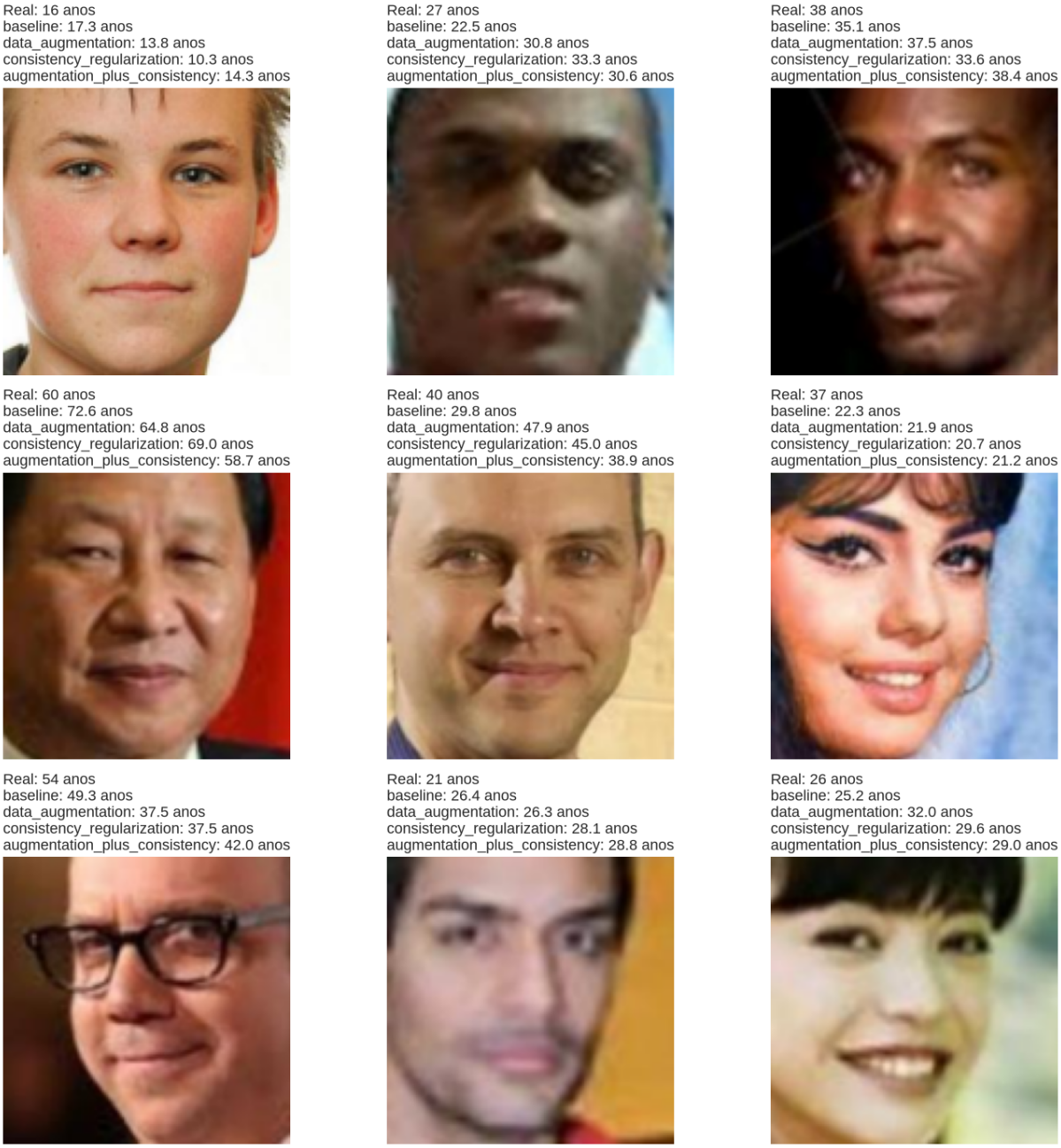



Figura 18: Exemplos de imagens do conjunto de teste. Para cada imagem são apresentados a idade real e as idades estimadas pelos modelos M1 (baseline), M2 (aumentação de dados), M3 (regularização de consistência) e M4 (combinação das duas estratégias).

Os resultados obtidos demonstram a importância da incorporação do princípio da invariância no treinamento da rede neural. Enquanto o modelo M1 foi treinado apenas com o erro supervisionado, os demais modelos exploraram diferentes mecanismos para tornar as predições mais robustas a pequenas variações nas imagens de entrada. Como consequência, todos os modelos baseados em invariância apresentaram desempenho superior ao baseline. Em particular, o modelo M4, que combina aumentação de dados e regularização de consistência, obteve os melhores resultados em todas as métricas analisadas. Compa-

rando as estratégias individualmente, observa-se que a regularização de consistência (M3) apresentou um desempenho ligeiramente superior ao obtido apenas com aumentação de dados (M2), embora ambas tenham contribuído para melhorar a capacidade de generalização da rede.

# Referências


[1] Charles R. Harris et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[2] John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9 (3):90–95, 2007.

[3] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

[4] Qunwei Li, Bhavya Kailkhura, Rushil Anirudh, Jize Zhang, Yi Zhou, Yingbin Liang, T. Yong Jin Han, and Pramod Varshney. Mr-gan: Manifold regularized generative adversarial networks for scientific data. *SIAM Journal on Mathematics of Data Science*, 3(4):1197–1222, 2021.

[5] Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035, 2019.